

Embedded systems task 02

Dead line: Saturday and Sunday 16th and 17th of May

(كل واحد في سكشنه و كل عشرة في القائمة في نص ساعة)

Bring your laptop to show me the code and the simulation, please make sure to understand what you're delivering and be ready for questions.

Deliverables:

Main.c file → includes main functions that implements the program core logic

Battery level indicator driver → .c and .h file (design it as you like)

Seven_segment_driver → provided, but it may need small edits according to your code

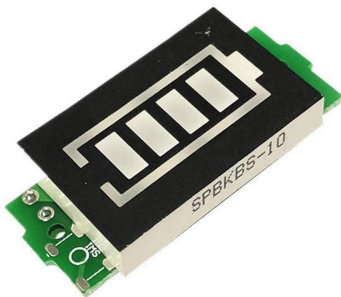
Motor_driver → .c , .h files with all the types defined and functions implemented

Gpio_driver → .c , .h files with all the types defined and functions implemented

Motor Control System (Code + Simulation)

System Components

- 3 Push Buttons (Switches)
- 1 DC Motor + Motor Driver (e.g., L298)
- 1 single-digit seven segment
- 1 double-digit seven segment module
- 1 3.7v Battery (use any controllable voltage source in simulation.
- 1 Battery level indicator (use 4 LEDs instead.check image below)



Make sure to use the drivers that we implemented together previously (gpio, seven_segment, motor_driver ...etc.)

1- create a driver for the battery indicator HW to abstract its functionality in simple function calls.

The indicator driver should have the following:

- Structure that contains its members (gpio pins, current battery level from 1 to 4, any other data needed)
- A function to initialize the module
- A function that takes the needed level as an input and applies it on the hardware

2- implement the following system

The system should behave as follows:

(please check the uml state machine diagram and activity diagram at the end of this document)

Initial State

- When the system starts:
 - Motor is **STOPPED**
 - Single digit 7-segment displays: **"S"** -> (same as **"5"**)
 - The system shall read the battery voltage using adc and show the value on the double-digit seven segment display.
 - The system shall map the battery voltage to a level(from 1 to 4) and display the corresponding level on the LED battery indicator
-

Switch 1 → Forward Direction

- When **Switch 1 is pressed**:
 - Motor rotates **forward**
 - Single digit 7-segment displays: **"F"**
 - When the button is released:
 - Motor **continues running forward** (no need to keep pressing)
-

Switch 2 → Backward Direction

- When **Switch 2 is pressed**:
 - Motor rotates **backward**
 - Single digit 7-segment displays: **"b"**
 - When the button is released:
 - Motor **continues running backward**
-

Switch 3 → Stop

- When **Switch 3 is pressed**:
 - Motor **stops**
 - Single digit 7-segment displays: **"S"**
-

- the motor driver HW is taking its power supply from the battery
 - The system shall monitor the battery level continuously (better to make it periodically each several seconds using a timer) and write the voltage readings on the double-digit seven segment display.
 - The system shall map the reading of the battery to a level and show that level on the battery indicator
 - Don't debounce the switches, just detect the rising edge and change the system state.
 - The system shall always remember the **last command** provided by the user (state-based system).
 - Only one state is active at a time:
 - Forward
 - Backward
 - Stopped
-

Important Notes

- Use the drivers software developed before in sections videos.
 - Use your **GPIO driver** for Reading switches (better to create a dedicated switches driver)
 - Do NOT write register names directly in `main.c`.
 - Don't panic if you have bugs or if the code is not working, all I care about is your understanding of what you're doing.
-

The following is 2 uml diagrams written in plantuml that represents the system please copy and paste the code(from @startuml to @enduml) to this website to visualize it: <https://www.plantuml.com/plantuml/uml/>

State machine Uml code:

@startuml

[*] --> Stopped

Stopped --> Forward : Switch1 Pressed

Stopped --> Backward : Switch2 Pressed

Forward --> Backward : Switch2 Pressed

Forward --> Stopped : Switch3 Pressed

Backward --> Forward : Switch1 Pressed

Backward --> Stopped : Switch3 Pressed

state Stopped : Motor = OFF\nDisplay = "S"

state Forward : Motor = FORWARD\nDisplay = "F"

```
state Backward : Motor = BACKWARD\nDisplay = "b"
```

```
@enduml
```

activity diagram that explains the program flow:

```
@startuml
```

```
start
```

```
:Initialize System;
```

```
:Initialize Drivers;
```

```
:Set Initial State (STOPPED);
```

```
fork
```

```
' =====
```

```
' Control Task (State Logic)
```

```
' =====
```

```
partition "Control Task" {
```

```
  while (System Running)
```

```
    :Read User Input;
```

```
    if (Switch1 Event?) then (Yes)
```

```
      :Set State = FORWARD;
```

```
      :Update Motor Direction;
```

```

        :Update Status Display ("F");
elseif (Switch2 Event?) then (Yes)
    :Set State = BACKWARD;
    :Update Motor Direction;
    :Update Status Display ("b");
elseif (Switch3 Event?) then (Yes)
    :Set State = STOPPED;
    :Stop Motor;
    :Update Status Display ("S");
endif
endwhile
}
fork again
' =====
' Battery Monitoring Task
' =====
partition "Battery Monitoring Task" {
    while (System Running)
        :Acquire Battery Reading (ADC);
        :Process Voltage Value;
        :Display Voltage (2-digit 7-seg);
    
```

```
        :Map Voltage to Level;  
        :Update Battery Indicator;  
    endwhile  
}  
end fork  
stop  
@enduml
```